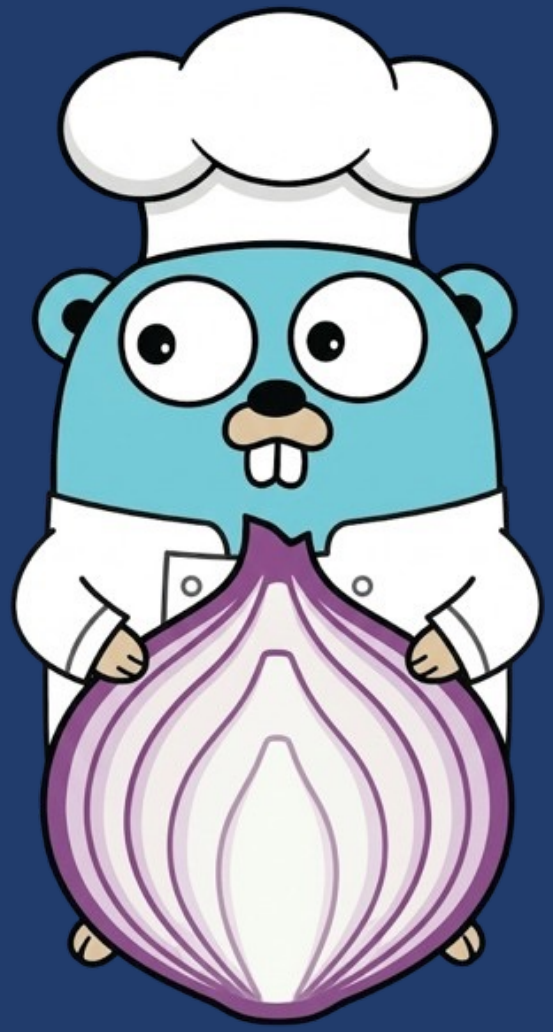
The Art of the Error: Building the Go-nion

Idiomatic errors and
(opinionated) best-practices



Go errors

The raw ingredients of a Go error

```
type error interface {
  Error() string
}
```

errors.New

```
var ErrNotFound = errors.New("not found")
```

Creates a distinct, static error value. Best used for **sentinel** errors.

fmt.Errorf

```
fmt.Errorf("user %q not found", name)
```

Allows dynamic, formatted strings. Enables error wrapping using the **%w** verb

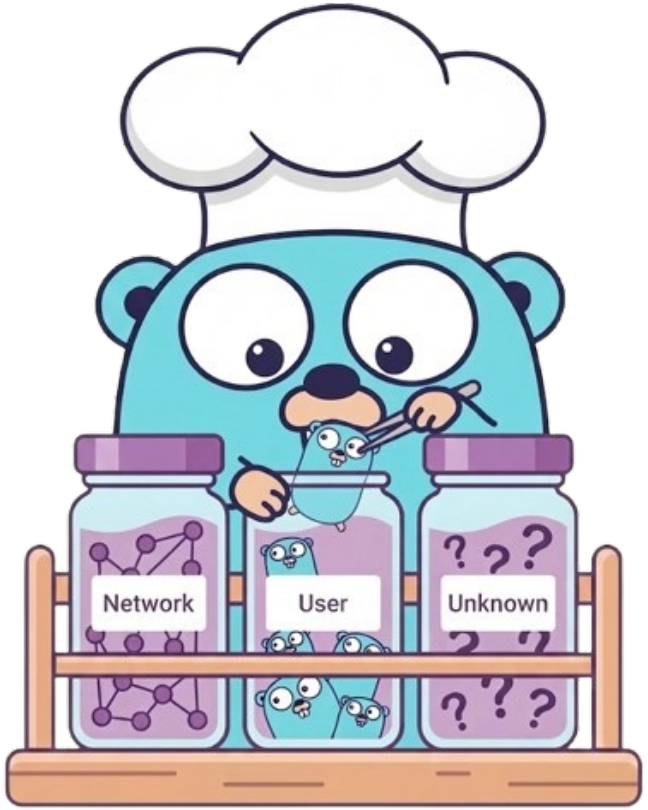
Structuring flavor with custom error types

```
type QueryError struct {
  message string
}
```

```
type QueryError struct {
  message string
  Cause ErrorCause
}

type ErrorCause int

const (
    NetworkError ErrorCause = iota + 1
    UserError
    UnknownError
)
```



Building the Go-nion: Adding layers of context

```
return fmt.Errorf("creating store: %w", err)
```

```
firstErr := errors.New("first error")
secondErr := fmt.Errorf("second error: %w", firstErr)
thirdErr := fmt.Errorf("third error: %w", secondErr)
```

```
error(*errors.errorString) *{
        s: "first error",}
```

```
error(*fmt.wrapError) *{
        msg: "third error: second error: first error",
        err: error(*fmt.wrapError) *{
                msg: "second error: first error",
                err: error(*errors.errorString) ...,},}
```

Second `err`

The core: Raw `err`
(e.g. DB failure)

The outer layer:
Added context

Peeling the layers and grouping the ingredients

Inspection (Peeling)

```
if errors.Is(err, infra.ErrNotExist) { ... }
```

```
if pathErr, ok := erros.AsType
     [*infra.ErrNotExist](err); ok { ... }
```

New Go 1.26 feature! Type-safe inspection

Joining (Grouping)

```
err := errors.Join(valErr1, valErr2)
```

```
error(*errors.joinError) *{
        errs: []error len: 2, cap: 2, [
                ...,
                ...,
        ],}
```

`errors` best practices

Add context, preserve the root.

Root cause destroyed

```
return fmt.Errorf("fetching user: %v", err)
```

Context added, root preserved

```
return fmt.Errorf("fetching user: %w", err)
```

Handle or `return`. Never both.

Delegate logging to client. Avoid repetition

```go
if err != nil {
  log.Printf("call service foo. %v", err)
  return fmt.Errorf("call service foo: %w", err)
}
```

```
2026/03/11 12:00:00 call service foo: connection refused
2026/03/11 12:00:00 fetch user: call service foo: connection
refused
2026/03/11 12:00:00 main process: fetch user: call service
foo: connection refused
```

```go
if err != nil {
  return fmt.Errorf("call service foo: %w", err)
}
```

```
2026/03/11 12:00:00 main process: fetch user: call service
foo: connection refused
```

Don’t leak the kitchen core.

Raw DB or filesystem errors can leak SQL paths, library versions or tokens.

Opaque errors

```
return fmt.Errorf("fetching user: %v", err)
```

The Split-Brain Pattern: Domain Boundaries

```
type SafeError struct {
  Code      string
  UserMsg   string
  Error     error
}
```

`Error`: stack trace used for logs

`UserMsg`: sanitized domain error served for public exposure (like API responses)

Stop the infrastructure error chain at the repository boundary.

Sanitize data before it crosses into the public dining room.

And remember, **`errors are values`**!

Would you return a DB DTO in the domain repository?

Resources

https://github.com/uber-go/guide/blob/master/style.md#errors

https://go.dev/blog/go1.13-errors

https://cs.opensource.google/go/go/+/refs/tags/go1.26.1:src/errors/errors.go

https://pkg.go.dev/errors@go1.26.1

https://www.datadoghq.com/blog/go-error-handling/

https://dave.cheney.net/2016/04/27/dont-just-check-errors-handle-them-gracefully

This blog is amazing! Has tons of articles regarding errors and other Go topics